C11 Subscription vs API : Patterns de Coût

Conception Intermediate

Comprendre quand chaque modèle de facturation est avantageux

Deux modèles fondamentalement différents

Subscription : vous payez un montant mensuel fixe et consommez dans la limite d'un budget de tokens. Le coût est prévisible, mais le budget est plafonné et les limites sont délibérément opaques.

API : facturation au token. Fixer et vérifier les plafonds pour le travail autonome ; compter tous les appels et reprises.

La distinction pratique : la subscription favorise les développeurs solo avec un usage quotidien régulier, l'API favorise les équipes, les pipelines CI/CD et les usages à fort volume.

Comment fonctionnent les limites subscription

Les subscriptions utilisent un modèle hybride : une fenêtre glissante de 5 heures et un cap hebdomadaire. Les deux s'appliquent simultanément. Le quota n'est pas publié comme une quantité fixe de tokens ; modèle, contexte et outils influencent la consommation.

Le quota ne suit pas un ratio de tarifs API. Le modèle, le contexte, les outils et la politique du compte influencent la consommation de l'abonnement. Consultez `/usage` et comparez des sessions réelles ; les tarifs API ci-dessous ne prédisent pas la vitesse d'épuisement du quota hebdomadaire.

Stratégies par plan

Plan	Recommandation
Pro	Choisir un modèle disponible pour la tâche ; surveiller `/usage`
Max 5x	Évaluer un routage explicite vers Sonnet face au modèle par défaut
Max 20x	Plus de liberté Opus, mais surveiller la consommation hebdomadaire
API	Haiku pour les tâches mécaniques, Sonnet pour le dev, Opus pour l'audit

Le pattern OpusPlan pour économiser

Avec un budget limité, comparez Opus pour planifier et Sonnet ou Haiku pour exécuter des tâches bornées. Mesurez les reprises et les résultats acceptés avant de conclure que ce routage économise de l'argent.

```
# Phase 1 : planification (Opus)
claude --model opus
"Analyse l'architecture et propose un plan de migration"

# Phase 2 : exécution (Sonnet/Haiku)
claude --model sonnet
"Implémente le plan selon les specs définies"
```

`/model opusplan` utilise précisément Opus pour la planification et Sonnet pour l'exécution ; il ne sélectionne pas Haiku.

Monitoring de consommation

```
/usage          # Coûts, limites et statistiques
/context        # Répartition des tokens

# Outil communautaire pour l'historique cross-sessions
ccusage         # Vue d'ensemble toutes périodes
ccusage daily   # Totaux quotidiens
ccusage monthly # Totaux mensuels
ccusage --breakdown # Par modèle
```

`/usage` fournit des métriques intégrées ; `ccusage` ajoute une analyse locale entre sessions. Aucune de ces estimations ne remplace une facture.

Quand l'API gagne

La subscription atteint ses limites dans trois cas : les équipes où plusieurs développeurs consomment simultanément, les pipelines CI/CD avec un volume de PRs élevé, et les projets avec des sessions longues quotidiennes qui épuisent le budget avant la fin de la semaine. Comparer le coût API total et les résultats acceptés avec le quota d'abonnement avant de changer.

Exemples avec budgets de tokens explicites

Tarifs API directs standard en USD, hors cache et fast mode :

Entrée / sortie	Sonnet 5.5	Opus 5.5
50K / 5K	$0.15	$0.30
150K / 20K	$0.50	$1.00

Le coût réel dépend de tous les appels et reprises.